

Register Number: 192425317

Name: M. Sahasra

CO1 – AT3

MLA0201 – Fundamentals of Machine Learning

Lab Experiments

Experiment 1: Probability Theory

A)

Title: Calculation of Class Probability Using the Breast Cancer Wisconsin Diagnostic Dataset

Aim:

To calculate the probability of each class (Malignant and Benign) using the Breast Cancer Wisconsin Diagnostic Dataset and use the computed probabilities to predict the class of a new data instance based on prior probability.

Theory:

Probability Theory is a branch of mathematics that deals with uncertainty and helps determine the likelihood of an event occurring. In Machine Learning, probability plays a vital role in classification, prediction, decision-making, and pattern recognition. It enables models to estimate how likely a data instance belongs to a particular class.

In classification problems, the probability of each class before considering the input features is called the Prior Probability. It is calculated by dividing the number of samples belonging to a particular class by the total number of samples in the dataset.

The Breast Cancer Wisconsin Diagnostic Dataset is widely used for binary classification problems. It contains medical measurements collected from breast cancer patients. Based on these measurements, each tumor is classified into one of two categories:

- Malignant (M): Cancerous tumor that can spread to other parts of the body.
- Benign (B): Non-cancerous tumor that does not spread.

The probability of each class is calculated using the formula:

$$P(Class) = \frac{\text{Number of samples belonging to the class}}{\text{Total number of samples}}$$

For this experiment,

$$P(M) = \frac{\text{Number of Malignant samples}}{\text{Total samples}}$$
$$P(B) = \frac{\text{Number of Benign samples}}{\text{Total samples}}$$

The class having the highest probability is considered the most likely prediction for a new data instance when only prior probabilities are considered. This concept forms the basis for many probabilistic machine learning algorithms such as Naïve Bayes.

Dataset name: Breast Cancer Wisconsin (Diagnostic) Data Set

Dataset link: <https://www.kaggle.com/datasets/uciml/breast-cancer-wisconsin-data>

Algorithm:

Step 1: Start the program.

Step 2: Import the required Python library (pandas) for handling the dataset.

Step 3: Load the Breast Cancer Wisconsin Diagnostic Dataset from the CSV file into a DataFrame.

Step 4: Display the first five rows of the dataset to verify that it has been loaded correctly.

Step 5: Calculate the total number of samples present in the dataset.

Step 6: Count the number of Malignant (M) samples.

Step 7: Count the number of Benign (B) samples.

Step 8: Calculate the probability of the Malignant class by dividing the number of Malignant samples by the total number of samples.

Step 9: Calculate the probability of the Benign class by dividing the number of Benign samples by the total number of samples.

Step 10: Display the calculated probabilities of both classes.

Step 11: Compare the probabilities of the Malignant and Benign classes.

Step 12: If the probability of Malignant is greater than the probability of Benign, predict the new data instance as Malignant. Otherwise, predict it as Benign.

Step 13: Display the predicted class.

Step 14: Stop the program.

Python Code:

```
[1]
✓ Os  1 import pandas as pd
2
3 # Load the dataset
4 df = pd.read_csv("data.csv")
5
6 # Display first five records
7 print("First Five Records of Dataset")
8 print(df.head())
9
10 # Calculate total samples
11 total_samples = len(df)
12
13 # Count class labels
14 class_counts = df["diagnosis"].value_counts()
15
16 malignant = class_counts["M"]
17 benign = class_counts["B"]
18
19 # Calculate probabilities
20 p_malignant = malignant / total_samples
21 p_benign = benign / total_samples
22
23 # Display counts
24 print("\nTotal Samples :", total_samples)
25 print("Malignant Samples :", malignant)
26 print("Benign Samples :", benign)
27
28 # Display probabilities
29 print("\nProbability of Malignant :", round(p_malignant,4))
30 print("Probability of Benign :", round(p_benign,4))
31
32 # Predict class using prior probability
33 if p_malignant > p_benign:
34     prediction = "Malignant"
35 else:
36     prediction = "Benign"
37
38 print("\nPredicted Class for New Data Instance :", prediction)
```

Sample Input:

Input Dataset: data.csv

Sample Output:

```
▼ ... First Five Records of Dataset
      id diagnosis radius_mean texture_mean perimeter_mean area_mean \
0      842302      M      17.99      10.38      122.80      1001.0
1      842517      M      20.57      17.77      132.90      1326.0
2      84300903     M      19.69      21.25      130.00      1203.0
3      84348301     M      11.42      20.38       77.58       386.1
4      84358402     M      20.29      14.34      135.10      1297.0

      smoothness_mean compactness_mean concavity_mean concave points_mean \
0      0.11840      0.27760      0.3001      0.14710
1      0.08474      0.07864      0.0869      0.07017
2      0.10960      0.15990      0.1974      0.12790
3      0.14250      0.28390      0.2414      0.10520
4      0.10030      0.13280      0.1980      0.10430

      ... texture_worst perimeter_worst area_worst smoothness_worst \
0      ...      17.33      184.60      2019.0      0.1622
1      ...      23.41      158.80      1956.0      0.1238
2      ...      25.53      152.50      1709.0      0.1444
3      ...      26.50      98.87      567.7      0.2098
4      ...      16.67      152.20      1575.0      0.1374

      compactness_worst concavity_worst concave points_worst symmetry_worst \
0      0.6656      0.7119      0.2654      0.4601
1      0.1866      0.2416      0.1860      0.2750
2      0.4245      0.4504      0.2430      0.3613
3      0.8663      0.6869      0.2575      0.6638
4      0.2050      0.4000      0.1625      0.2364

      fractal_dimension_worst Unnamed: 32
0      0.11890      NaN
1      0.08902      NaN
2      0.08758      NaN
3      0.17300      NaN
4      0.07678      NaN

[5 rows x 33 columns]

Total Samples : 569
Malignant Samples : 212
Benign Samples : 357

Probability of Malignant : 0.3726
Probability of Benign : 0.6274

Predicted Class for New Data Instance : Benign
```

Result:

The Breast Cancer Wisconsin Diagnostic Dataset was successfully loaded, and the probabilities of the Malignant and Benign classes were calculated. The probability of the Benign class (0.6274) was found to be greater than the probability of the Malignant class

(0.3726). Based on the computed prior probabilities, the new data instance was predicted as Benign.

Conclusion:

This experiment successfully demonstrated the application of Probability Theory in Machine Learning. The prior probabilities of the Malignant and Benign classes were calculated using the Breast Cancer Wisconsin Diagnostic Dataset. Since the Benign class had a higher probability than the Malignant class, it was selected as the predicted class for a new data instance. This experiment illustrates how probability-based decision-making forms the foundation of many machine learning algorithms, especially probabilistic classifiers such as the Naïve Bayes algorithm.

Experiment 2: Bayes Decision Theory

A)

Title: Classification of SMS Messages Using Bayes Decision Theory

Aim:

To implement Bayes Decision Theory using the SMS Spam Collection Dataset, calculate the posterior probability of a given SMS message, and classify it as Spam or Ham (Not Spam).

Theory:

Bayes Decision Theory is a statistical approach used in Machine Learning for making decisions under uncertainty. It is based on Bayes' Theorem, which calculates the probability of an event occurring based on prior knowledge and observed evidence.

In Machine Learning, Bayes Decision Theory is commonly used for classification problems. It predicts the class of a new data instance by calculating the posterior probability of each class and selecting the class with the highest probability.

The mathematical formula of Bayes' Theorem is:

$$P(A | B) = \frac{P(B | A) \times P(A)}{P(B)}$$

Where:

- $P(A|B)$ = Posterior Probability (Probability of class A after observing data B)
- $P(B|A)$ = Likelihood (Probability of observing data B when class A is true)

- $P(A)$ = Prior Probability (Initial probability of class A)
- $P(B)$ = Evidence (Probability of observing the data)

Naïve Bayes Classifier:

The Naïve Bayes algorithm is based on Bayes' Theorem and assumes that all input features are independent of each other. Although this assumption is rarely true in real-world data, Naïve Bayes performs well in text classification tasks such as email filtering and SMS spam detection.

SMS Spam Collection Dataset:

The SMS Spam Collection Dataset contains thousands of SMS messages labeled as:

- Spam: Unwanted promotional or fraudulent messages.
- Ham: Genuine or legitimate messages.

Examples:

Spam:

- Congratulations! You have won a free prize.
- Claim your reward now.

Ham:

- Are you coming to class today?
- Let's meet at 5 PM.

During training, the model learns the probability of words appearing in Spam and Ham messages. When a new SMS is given, the classifier calculates the posterior probability for both classes and predicts the class with the higher probability.

Bayes Decision Theory is widely used in:

- Email Spam Detection
- SMS Spam Filtering
- Document Classification
- Sentiment Analysis

- Medical Diagnosis
- Fraud Detection

Dataset name: SMS Spam Collection Dataset

Dataset link: <https://www.kaggle.com/datasets/uciml/sms-spam-collection-dataset>

Algorithm:

Step 1: Start the program.

Step 2: Import the required Python libraries such as Pandas, CountVectorizer, Train-Test Split, and Multinomial Naïve Bayes.

Step 3: Load the SMS Spam Collection Dataset into a DataFrame.

Step 4: Select only the required columns (Label and Message).

Step 5: Rename the columns as label and message for easy understanding.

Step 6: Separate the dataset into input messages (X) and target labels (Y).

Step 7: Convert the text messages into numerical feature vectors using the CountVectorizer technique.

Step 8: Split the dataset into training data and testing data.

Step 9: Train the Multinomial Naïve Bayes classifier using the training data.

Step 10: Enter a new SMS message for testing.

Step 11: Convert the new message into a numerical vector using the same CountVectorizer.

Step 12: Use the trained Naïve Bayes model to predict whether the message is Spam or Ham.

Step 13: Display the predicted class.

Step 14: Stop the program.

Python Code:

```
[2]
✓ 1s
1 import pandas as pd
2 from sklearn.model_selection import train_test_split
3 from sklearn.feature_extraction.text import CountVectorizer
4 from sklearn.naive_bayes import MultinomialNB
5
6 # Load dataset
7 df = pd.read_csv("spam.csv", encoding="latin-1")
8
9 # Keep only required columns
10 df = df[['v1', 'v2']]
11 df.columns = ['label', 'message']
12
13 # Display first five records
14 print("First Five Records")
15 print(df.head())
16
17 # Separate features and target
18 X = df['message']
19 y = df['label']
20
21 # Convert text into numerical vectors
22 vectorizer = CountVectorizer()
23 X = vectorizer.fit_transform(X)
24
25 # Split dataset
26 X_train, X_test, y_train, y_test = train_test_split(
27     X, y, test_size=0.2, random_state=42
28 )
29
30 # Train Naive Bayes model
31 model = MultinomialNB()
32 model.fit(X_train, y_train)
33
34 # Test message
35 new_message = ["Congratulations! You have won a free prize. Click here to claim now."]
36
37 # Convert new message into vector
38 new_vector = vectorizer.transform(new_message)
39
40 # Predict
41 prediction = model.predict(new_vector)
42
43 print("\nTest Message:")
44 print(new_message[0])
45
46 print("\nPredicted Class:", prediction[0])
```

Sample Input:

Input Dataset: spam.csv

Sample Output:

```
▼ ... First Five Records
      label                                message
0   ham  Go until jurong point, crazy.. Available only ...
1   ham                                     Ok lar... Joking wif u oni...
2  spam  Free entry in 2 a wkly comp to win FA Cup fina...
3   ham  U dun say so early hor... U c already then say...
4   ham  Nah I don't think he goes to usf, he lives aro...

Test Message:
Congratulations! You have won a free prize. Click here to claim now.

Predicted Class: spam
```

Result:

The SMS Spam Collection Dataset was successfully loaded and processed. The SMS messages were converted into numerical feature vectors using the CountVectorizer technique, and a Multinomial Naïve Bayes classifier was trained using the training data. When a new SMS message was provided as input, the classifier correctly calculated the posterior probability and classified the message as Spam.

Conclusion:

This experiment successfully demonstrated the application of Bayes Decision Theory in Machine Learning. The Multinomial Naïve Bayes algorithm was used to classify SMS messages into Spam and Ham by calculating posterior probabilities based on Bayes' Theorem. The experiment showed that Naïve Bayes is an efficient and accurate algorithm for text classification tasks such as spam filtering. It is widely used in real-world applications, including email filtering, SMS spam detection, document categorization, and sentiment analysis.

Experiment 3: Information Theory

A)

Title: Calculation of Entropy and Information Gain Using the Play Tennis Dataset

Aim:

To calculate the entropy and information gain for all input attributes using the Play Tennis Dataset and identify the attribute with the highest information gain for decision tree construction.

Theory:

Information Theory is a mathematical concept introduced by Claude Shannon to measure the amount of uncertainty or impurity present in a dataset. In Machine Learning, Information Theory is mainly used in Decision Tree algorithms such as ID3, C4.5, and C5.0 to determine the best attribute for splitting the dataset.

The two important concepts used in Information Theory are Entropy and Information Gain.

Entropy:

Entropy measures the uncertainty or randomness in a dataset. If all records belong to the same class, the entropy is 0, indicating no uncertainty. If the records are evenly distributed among different classes, the entropy is maximum, indicating high uncertainty.

The formula for entropy is:

$$Entropy(S) = - \sum_{i=1}^n p_i \log_2(p_i)$$

Where:

- S = Dataset
- p_i = Probability of class i

Information Gain:

Information Gain measures the reduction in entropy after splitting the dataset based on a particular attribute. It helps determine which attribute best separates the data into distinct classes.

The formula is:

$$Information\ Gain = Entropy(S) - \sum \frac{|S_v|}{|S|} \times Entropy(S_v)$$

Where:

- S = Original dataset
- S_v = Subset after splitting based on an attribute

The attribute with the highest Information Gain is selected as the root node of the Decision Tree because it provides the maximum reduction in uncertainty.

Play Tennis Dataset:

The Play Tennis dataset is a simple dataset used to demonstrate Decision Tree construction. It contains weather-related attributes such as:

- Outlook
- Temperature
- Humidity
- Wind

Target Attribute:

- Play Tennis
 - Yes
 - No

By calculating the entropy and information gain of each attribute, the best attribute for classification can be identified.

Dataset name: Play tennis

Dataset link: <https://www.kaggle.com/datasets/fredericobreno/play-tennis>

Algorithm:

Step 1: Start the program.

Step 2: Import the required Python libraries such as Pandas, LabelEncoder, and Mutual Information functions.

Step 3: Load the Play Tennis dataset into a DataFrame.

Step 4: Display the first few rows of the dataset to verify that it has been loaded correctly.

Step 5: Separate the input attributes (Outlook, Temperature, Humidity, Wind) from the target attribute (Play Tennis).

Step 6: Convert all categorical values into numerical values using Label Encoding because machine learning algorithms require numerical input.

Step 7: Calculate the Information Gain (Mutual Information) for each input attribute with respect to the target class.

Step 8: Display the Information Gain values of all attributes.

Step 9: Compare the Information Gain values.

Step 10: Identify the attribute having the highest Information Gain.

Step 11: Display the best attribute for constructing the Decision Tree.

Step 12: Build a Decision Tree using the Entropy criterion.

Step 13: Stop the program.

Python Code:

```
[3]
✓ Os
1 import pandas as pd
2 from sklearn.preprocessing import LabelEncoder
3 from sklearn.feature_selection import mutual_info_classif
4 from sklearn.tree import DecisionTreeClassifier
5
6 # Load dataset
7 df = pd.read_csv("play_tennis.csv")
8
9 # Display dataset
10 print("First Five Records")
11 print(df.head())
12
13 # Separate features and target
14 X = df.iloc[:, :-1]
15 y = df.iloc[:, -1]
16
17 # Encode categorical values
18 encoder = LabelEncoder()
19
20 for column in X.columns:
21     X[column] = encoder.fit_transform(X[column])
22
23 y = encoder.fit_transform(y)
24
25 # Calculate Information Gain
26 info_gain = mutual_info_classif(X, y, random_state=42)
27
28 print("\nInformation Gain of Each Attribute")
29
30 for feature, value in zip(X.columns, info_gain):
31     print(feature, ":", round(value,4))
32
33 # Best attribute
34 best_attribute = X.columns[info_gain.argmax()]
35
36 print("\nBest Attribute :", best_attribute)
37
38 # Build Decision Tree
39 model = DecisionTreeClassifier(criterion="entropy")
40
41 model.fit(X, y)
42
43 print("\nDecision Tree Model Created Successfully")
```

Sample Input:

Input Dataset: play_tennis.csv

Sample Output:

▼	...	First Five Records					
	day	outlook	temp	humidity	wind	play	
	0	D1	Sunny	Hot	High	Weak	No
	1	D2	Sunny	Hot	High	Strong	No
	2	D3	Overcast	Hot	High	Weak	Yes
	3	D4	Rain	Mild	High	Weak	Yes
	4	D5	Rain	Cool	Normal	Weak	Yes
		Information Gain of Each Attribute					
		day : 0.0					
		outlook : 0.0795					
		temp : 0.0124					
		humidity : 0.0523					
		wind : 0.0					
		Best Attribute : outlook					
		Decision Tree Model Created Successfully					

Result:

The Play Tennis dataset was successfully loaded and processed. The entropy and information gain of all input attributes were calculated. Among all the attributes, Outlook had the highest Information Gain. Therefore, Outlook was selected as the best attribute for constructing the Decision Tree. The Decision Tree model was successfully built using the Entropy criterion.

Conclusion:

This experiment successfully demonstrated the application of Information Theory in Machine Learning. Entropy was used to measure the uncertainty in the dataset, while Information Gain was used to determine the most informative attribute for classification. The attribute Outlook had the highest Information Gain, indicating that it is the best choice for splitting the dataset. This experiment shows how Decision Tree algorithms use Information Theory to make accurate classification decisions and reduce uncertainty in data.